

FASE 1: EL PROBLEMA DE LOS 3 FOCOS

1.1. Fundamentos del Diseño Funcional e Inmutabilidad Absoluta

El desarrollo del núcleo lógico para el Sistema de Semáforos Inteligentes se estructuró bajo las pautas estrictas del paradigma funcional puro, rompiendo de forma categórica con los esquemas de la programación imperativa tradicional. En el diseño implementado, se erradicó por completo el uso de variables globales mutables (`defparameter`, `defvar`) y operadores de asignación destructiva que provoquen efectos colaterales en la memoria (`setq`, `setf`).

Para modelar un entorno físico intrínsecamente cambiante como un semáforo sin alterar el estado de la memoria, se recurrió a la representación algebraica de los estados mediante símbolos literales inmutables: `'en-rojo`, `'en-verde`, `'en-amarillo` y `'amarillo-intermitente`. El paso del tiempo y las transiciones secuenciales no modifican una celda de memoria existente; en su lugar, el flujo de datos y el estado actual del tráfico y del tiempo viajan de manera explícita y exclusiva a través de los argumentos de las funciones, respetando la secuencia vial real de seguridad (salto directo de rojo a verde, y uso de la fase amarilla estrictamente como advertencia preventiva de frenado posterior al verde). Cada ciclo del sistema evalúa las precondiciones de entrada y computa una nueva estructura de datos de salida totalmente aislada, garantizando la predictibilidad matemática y la consistencia referencial del software.

1.2. Análisis Taxonómico de las Funciones Núcleo

A continuación, se detalla la clasificación taxonómica obligatoria de las funciones desarrolladas en Common Lisp para dar cumplimiento a los requerimientos 1 al 6 y la extensión de intermitencia de la iteración 2:

I. Función transición

- Naturaleza: Pura. Dado un color actual y una orden de cambio equivalentes, el resultado devuelto es determinista y no genera alteraciones colaterales en el entorno de ejecución.
- Estrategia de Control: Función Predicado / Condicional Declarativo. Evalúa mediante cláusulas lógicas con el macro `cond` la validez de la secuencia de tráfico vehicular.
- Impacto en Memoria: No destructiva. Instancia y retorna una nueva estructura de lista en cada invocación empleando la primitiva `list`. El diseño computa y proyecta en el primer elemento de la lista el símbolo del nuevo estado resultante inmutable (v.g., `'en-verde`) en lugar del estado de origen, garantizando la correcta transición de fases exigida en la especificación de la cátedra sin alterar los punteros de los argumentos de entrada.

II. Función timer

- Naturaleza: Pura. Recibe de forma determinista una marca de tiempo Unix (Epoch) y mapea matemáticamente el color activo correspondiente al período relativo.

- Estrategia de Control: Composición funcional y aritmética plana. Emplea una operación de aritmética de módulo sobre el tiempo acumulado del ciclo total para deducir la fase actual.
- Impacto en Memoria: No destructiva. Genera un símbolo como retorno independiente, preservando intacto el valor entero del timestamp ingresado.

III. Función registrar-cambio

- Naturaleza: Impura. Su propósito explícito es interactuar con el subsistema de Entrada/Salida (I/O) del entorno, generando un efecto secundario visible al escribir cadenas formateadas en la consola de auditoría forense.
- Estrategia de Control: Función de flujo secuencial / Mapeo de salida.
- Impacto en Memoria: No destructiva. No altera estructuras de datos del programa; su salida se consume directamente en el stream de la terminal.

IV. Funciones de Análisis de Ciclos (duración-ciclo y recomendacion-ciclo)

- Naturaleza: Pura. Realizan cálculos y evaluaciones estrictamente deterministas sobre rangos estándar de ingeniería de tráfico.
- Estrategia de Control: Condicional Declarativo.
- Impacto en Memoria: No destructiva. Retornan constantes numéricas o cadenas de texto literales sin modificar parámetros previos.

1.3. Estrategias de Optimización y Simulación del Tiempo (TCO)

Para ejecutar la simulación del paso del tiempo y dar cumplimiento al Requerimiento 7 (Aseguramiento de la Calidad) sin recurrir a estructuras de control imperativas prohibidas (while, loop), se diseñó la función motora simular-linea-tiempo aplicando Recursividad de Cola (Tail Recursion).

En un esquema de recursividad simple, cada llamada descendente retiene un contexto operativo en la pila de ejecución esperando resolver una operación posterior (por ejemplo, acumular o evaluar un dato rezagado), lo que genera un crecimiento lineal de la memoria de tamaño $O(n)$. Frente a simulaciones de tiempos prolongados, este comportamiento agota los recursos del entorno y gatilla un desborde crítico.

Nuestra solución sitúa la llamada recursiva a simular-linea-tiempo como la última expresión absoluta a ser evaluada en el cuerpo de la función. Esto permite que el compilador de Common Lisp aplique de forma nativa la Optimización de Llamada de Cola (TCO, Tail Call Optimization), transformando internamente la cascada de llamadas en un salto iterativo a bajo nivel. Como consecuencia, el marco de la pila (stack frame) se reutiliza indefinidamente, reduciendo el consumo de espacio en memoria a una constante $O(1)$ y permitiendo procesar flujos de tiempo extensos con total seguridad y estabilidad en el sistema embebido.

1.4. Bitácora de Depuración y Errores Conceptuales

Durante la transición hacia el pensamiento funcional, el equipo experimentó y resolvió los siguientes cinco fallos de carácter conceptual:

Bug 1: Control Stack Exhausted por Recursión Ineficiente

- Síntoma: Al ejecutar pruebas unitarias con marcas de tiempo representativas de varias horas de operación, el entorno de desarrollo (SBCL) abortaba abruptamente la ejecución arrojando la excepción: Control stack exhausted.
- Causa Conceptual: El prototipo inicial de la función de simulación incrementaba el tiempo sumando un valor de manera diferida fuera de la llamada recursiva, impidiendo la liberación de la pila. El programa acumulaba cada segundo transcurrido como un marco pendiente en memoria.
- Diagnóstico y Corrección: Se reestructuró la lógica eliminando cualquier cálculo diferido. Los parámetros incrementales (tales como (+ timestamp-actual 1)) se inyectaron directamente dentro del paso de argumentos de la función recursiva, garantizando una llamada de cola pura optimizable por el compilador mediante la reutilización del marco operativo.

Bug 2: Violación de Inmutabilidad con Bloques Locales

- Síntoma: Advertencias del compilador indicando efectos secundarios no controlados o comportamientos inconsistentes al intentar actualizar el valor de la fase actual conforme avanzaba el tiempo.
- Causa Conceptual: Arrastre de vicios del paradigma imperativo. Se intentó forzar el uso del bloque de asignación local let de manera que simulara una variable tradicional que mutaba su estado interno a través de la ejecución de un subproceso operativo.
- Diagnóstico y Corrección: Se asimiló el principio de que en este paradigma los identificadores representan ligaduras matemáticas inmutables. Se eliminó el bloque de asignación mutable interna y se delegó la actualización de las fases al motor evaluador de la función pura timer, cuyo resultado alimenta de forma limpia y directa las transiciones secuenciales entre llamadas continuas.

Bug 3: Evaluación Errónea de Símbolos (Unbound Variable)

- Síntoma: Al ingresar pruebas manuales en el REPL como (transicion en-rojo verde), el evaluador interrumpía el hilo levantando una excepción del tipo: Cell-error: The variable EN-ROJO is unbound.
- Causa Conceptual: Confusión conceptual entre un símbolo interpretado como dato y un símbolo evaluado como variable. El lector de Lisp intentaba buscar el valor en memoria apuntado por la palabra en-rojo en lugar de tratarla como un átomo identificador único en sí mismo.
- Diagnóstico y Corrección: Se aplicó el mecanismo de cotización anteponiendo una comilla simple ('en-rojo). Esto le instruye explícitamente al entorno que suspenda la evaluación regular de variables y procese el símbolo como un valor literal inmutable para las comparaciones de estados del semáforo.

Bug 4: Acoplamiento de Efectos Secundarios en Funciones Matemáticas

- Síntoma: Dificultad extrema para realizar pruebas unitarias automatizadas sobre la función timer, ya que cada cálculo matemático provocaba impresiones de logs desordenadas y asíncronas en la pantalla de depuración.

- Causa Conceptual: Se corrompió la pureza referencial al mezclar lógica de procesamiento matemático (el cálculo del color según el residuo del tiempo Unix) con acciones imperativas de Entrada/Salida (el uso del macro format para interactuar con el flujo de la consola).
- Diagnóstico y Corrección: Se aplicó el principio de separación estricta de responsabilidades funcionales. Se depuró la función timer removiendo toda lógica de impresión para clasificarla como una función puramente matemática y determinista. El mecanismo de logging de auditoría forense fue aislado por completo dentro de la función específica impura registrar-cambio.

Bug 5: Error de Modelado de Dominio en la Secuenciación de Tráfico Urbano

- Síntoma: Al simular los flujos de tránsito en las pruebas de integración, se detectó que el sistema permitía transiciones lineales ingenuas (como Rojo → Amarillo → Verde) que en la normativa de vialidad real representan un riesgo crítico de colisión y frenado intempestivo en las intersecciones.
- Causa Conceptual: Una comprensión incompleta de las reglas de negocio del dominio físico. Se asumió una máquina de estados puramente circular en el prototipo inicial, en lugar de modelar las restricciones de seguridad vial donde el amarillo actúa exclusivamente como fase de descompresión y advertencia posterior al verde.
- Diagnóstico y Corrección: Se refactorizó la función condicional transicion restringiendo los caminos válidos: el sistema ahora solo convalida el salto directo de en-rojo al estado de flujo verde, y confina el uso del amarillo como el nexó restrictivo previo al regreso al estado de detención absoluta (rojo). Cualquier estímulo externo que intente violar esta secuencia segura es interceptado por la cláusula por defecto (t), manteniendo el semáforo en su estado actual y arrojando 'accion-por-defecto'.